The Tool Count Cliff: A Small Model's Categorical Collapse Where Larger Models Merely Decay

Ricardo Martínez

Cloud Security & Applied AI

November 2026

Abstract

A 9.7B open-weight model, given a single synthetic tool, called it correctly every time and completed a full tool-result round trip. The identical model, same backend, same sampling settings, wired into a real agent's production toolset of over twenty tools, never attempted a real tool call again across repeated turns on two independent surfaces — it wrote the shell command as prose instead of invoking it. Lowering sampling temperature fixed the isolated case outright and left the production case completely unchanged. Public benchmarks already document tool-count degradation in mid-size and frontier models as a gradual accuracy decline; this field case adds a small model's version of the same axis, which looks less like a decline and more like a step function. The practical consequence: certifying a model's tool-calling at one tool count says nothing reliable about a different tool count, and the gap is not always a slope.

1 Two Measurements of the Same Model

The first test is the one almost every agent build starts with: give the model one tool, a clear instruction to use it, and confirm it does. A 9.7B open-weight model passed this cleanly — a correctly typed tool call, every time, followed by a clean reinjected tool-result round trip. By that test, the model is capable of tool-calling.

The second test used the identical model, the identical backend, the identical sampling settings, wired into a real personal-agent platform's actual deployed toolset: terminal access, file operations, web search, vision, image generation, speech, browser automation, skill loading, scheduled jobs, and messaging, composited into toolsets exceeding twenty individual tools. Across two independent entry surfaces — a chat gateway and a direct command-line invocation — the model answered in plain text every single time. Both surfaces' own request logs showed `api_calls=1`: the model never even reached the tool-call turn a real invocation requires. It narrated a shell command as prose instead of calling the terminal tool that was sitting right there in its payload.

	Isolated test	**Production test**
Tools offered	1 (synthetic)	20+ (real toolset)
Tool call attempted	Yes, every turn	No, zero turns
`api_calls`	2+ (real round trip)	1 (text only)
Surfaces tested	1	2, independently

Table 1: Same model, same backend, same sampling settings. The only variable that changed between the two columns is how many tools were in the payload.

2 The Temperature Hypothesis, Half Right

The deployment's default sampling parameters — temperature 1.0, presence penalty 1.5 — were the obvious first suspect. Those settings favor novel, high-entropy continuations, which is close to the opposite of what deterministic tool-calling needs: a model that reliably reproduces the exact structured call the schema asks for, not an original one.

A low-temperature (0.1), zero-presence-penalty variant of the same model was built and tested directly against the model-serving API. It fixed the isolated case outright: a correctly typed tool call, verified end to end through a full reinjected-result round trip, with no ambiguity about whether it was a fluke. Wired

into the identical production toolset under the identical low-temperature variant, on both of the same two independent surfaces, the model still never attempted a single tool call.

The hypothesis was not wrong. It was incomplete. Sampling settings mattered enormously for the one-tool case and explained none of the production case. Whatever gates tool-calling once the toolset grows past some threshold, it is not primarily a sampling problem for this model.

3 What the Public Literature Already Shows

Tool-count degradation in LLM agents is a documented phenomenon, reported unevenly across model sizes and benchmark designs:

- LangChain's ReAct benchmark found accuracy held for `o1` but fell sharply for `o3-mini` and to near zero for `GPT-4o` once domain count passed roughly seven, as reported in StackOne's tool-count-ceiling analysis.
- Maxim AI's evaluation of five models across GitHub and Notion tool sets found that cutting available tools from 48 to 25 improved accuracy for every model tested, including Claude Sonnet 4 (66.67% to 73.33%).
- Anthropic's own reporting on its Tool Search Tool shows Claude Opus 4.5 moving from 79.5% to 88.1% on an internal benchmark once tools are discovered on demand rather than all declared up front — a vendor figure, on the vendor's own benchmark.
- TSCG (May 2026) reports restoring Phi-4 14B from 0% to 84.4% accuracy at 20 tools through schema compilation: a mid-size model hitting a wall at a comparable tool count to the one measured here for a smaller model.

A practitioner analysis of this body of work states the gap plainly: no public benchmark cleanly publishes the size-decay axis — hold the model fixed, vary tool count, plot accuracy. Most published comparisons hold tool count fixed and vary the model, or hold the model fixed and compare exactly two tool counts chosen to support a specific product claim, not a curve across sizes.

4 What a Cliff Looks Like, Not a Slope

The benchmarks above describe decay: accuracy drops as tool count rises, but for models capable of tool-calling at all, it rarely reaches zero. The 9.7B model measured here did not decay between the two tested points — it was reliable at one and made zero real tool calls at the other, under two sampling regimes and two independent surfaces.

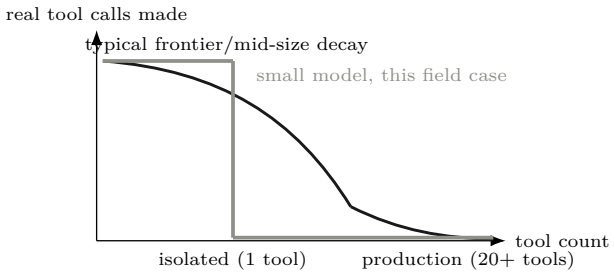


Figure 1: Illustrative, not measured on a continuous axis: the documented frontier/mid-size pattern is a decline across the curve; the two points measured for the small model in this case are a step, not a slope.

This distinction matters for anyone deciding how much weight to put on a passed smoke test. A gradual decay can be budgeted for — route the hardest calls to a bigger model, trim the toolset, accept a known error rate. A categorical stop cannot be budgeted the same way: past whatever threshold triggers it, the small model is not "somewhat less reliable," it is not participating in the tool-calling contract at all, silently, with no error raised anywhere.

5 Design Principle

Certify a model's tool-calling capability at the tool count it will actually run with in production, not at whatever count is cheapest to write a smoke test for. A pass at $N{=}1$ is evidence about $N{=}1$. It is not evidence about $N{=}20$, and for a small model, the two can be categorically different outcomes rather than nearby points on a curve.

The minimal pattern in `patterns/01_isolated_vs_production_to` makes this concrete: a certification routine that only accepts a model as tool-capable if it demonstrates that capability at the real deployment's tool count, with intermediate probe sizes recorded so a cliff — unlike a decay — is visible as soon as it appears, not discovered later in production.

Methodological Note

This is a single field case, not a controlled study across multiple models or tool sets. The two tool counts, the sampling settings, and the `api_calls` figures cited are real measurements from the same system across two independent entry surfaces, not simulated. No causal claim is made about mechanism — attention dilution, instruction dilution from a longer system prompt, or something specific to this model family — only that the isolated-tool-count test and the production-tool-count test produced categorically different results for the same model under the same settings.

References

[1] StackOne. *The Agent Tool Count Ceiling*. stackone.com/blog/agent-tool-count-ceiling.

[2] Maxim AI. *Tool Chaos No More: How We're Measuring Model-Tool Accuracy in the Age of MCP*. getmaxim.ai/blog.

[3] BenchmarkList. *TSCG: Deterministic Tool-Schema Compilation for Agentic LLM Deployments*. benchmarklist.com.

[4] HackerNoon. *AI Agents Pick the Wrong Tool More Often as You Add Tools, and Research Shows Why*. hackernoon.com.